

频域分解增强的轻量级 IMU 手势识别系统

张倚萌，赵霖霏，孙蝶梦 2026 年 6 月

摘要—本文面向芯原 VeriHealthi QEMU SDK 平台，实现了一套可实时运行的惯性测量单元手势识别系统。系统在 RISC-V nuclei_evalsoc 架构和 FreeRTOS 实时操作系统上处理 50 Hz 六轴 IMU 数据，识别双指互点、握拳、抬腕和放下四类手势。算法将 77 维时域统计量与 8 维滤波器组能量特征合成为 85 维输入；全局五分类网络先判断候选类别，专用二分类网络再区分双指互点与握拳，短序列校验利用最近 8 个窗口的分类置信度抑制孤立误判，质心距离校验用于排除小手势离群样本。上述输出由状态机整合为离散事件，并通过 UART 和 BLE GATT 服务发送。在 QEMU 验证序列的 87 个标注事件上，系统取得 macro F1=0.9896，其中握拳、抬腕和放下的 F1 指标均为 1.0000，双指互点的 F1 指标为 0.9583。CRC-8/SMBus 输出值为 0x37。最终工程 Flash 占用 556.3 KB，RAM 占用 106.3 KB，模型参数约 48 KB。

关键词：IMU 手势识别，频域分解，滤波器组特征，时序卷积网络，BLE GATT，嵌入式系统，RISC-V，FreeRTOS

I. 引言

近年来，智能可穿戴设备的手势识别技术在人机交互、健康监测和智能控制等领域得到广泛研究 [2], [1], [3]。相比视觉方案，IMU 方案功耗低、隐私风险小且不受光照条件影响，适合部署在智能手表、智能手环等可穿戴设备上。

不过，在嵌入式平台上实现高精度实时手势识别仍面临多个工程难题：嵌入式设备的计算资源和存储空间有限，难以承载大规模深度学习模型；IMU 信号噪声较大、用户间差异明显、手势类间相似度高；实时系统还要求算法的延迟和执行时间具有确定性。

芯原 VeriHealthi 竞赛提供了上述问题的标准化验证平台。竞赛要求参赛者在 VeriHealthi QEMU SDK 上，利用三轴陀螺仪和三轴加速度计数据，实时识别 pinch、clench、up 和 down 四类手势，分别对应双指互点、握拳、抬腕和放下，并开发 BLE GATT 服务以支持识别结果的无线传输。

A. 研究目标与贡献

本文围绕该目标完成了以下工作：

- 1) **频域分解特征：**设计了包含 77 维统计特征和 8 维滤波器组特征的 85 维输入。滤波器组借鉴文献 [1] 中的信号分解思想，通过低通、高通和中频带通滤波器刻画原始 IMU 信号的不同频段能量，提升对 pinch 等微小手势的辨识能力。
- 2) **小手势精细判别：**在全局五分类网络之外，引入 pinch/clench 二分类网络、8 窗口时序校验和质心距离校验。全局网络负责确定候选手势类型，后续校验只在小手势候选上参与决策，从而降低 pinch 与 clench 之间的混淆。
- 3) **状态机决策逻辑：**设计了包含小手势聚合、大手势连续确认、不应期控制等机制的状态机，在时间维度上实现事件级去重和抗抖动。
- 4) **BLE GATT 服务实现：**基于 Zephyr BLE 协议栈，实现了 128 位 UUID 自定义 GATT 服务，支持 Read、Notify 和 Indicate 三种属性，满足手势结果无线传输需求 [5]。
- 5) **低资源占用：**整个系统 Flash 占用约 556 KB、RAM 占用约 107 KB，远低于平台 3 MB / 256 KB 的上限。

目 录

I	引言	1
I-A	研究目标与贡献	1
II	方案介绍	2
II-A	系统总体架构	2
II-B	平台规格与约束	2
II-C	数据集概况	3
II-D	评估指标	4
III	算法设计	4
III-A	数据预处理与滑动窗口	4
III-A1	原始数据格式转换	4
III-A2	滑动窗口机制	4
III-B	特征工程	5

III-B1	77 维统计特征	5	VI-D	Python 算法验证结果	17
III-B2	8 维滤波器组特征	5	VI-E	资源使用分析	17
III-C	原始 IMU 信号分析	6	VI-E1	代码与数据段大小	17
III-D	分类推理框架	6	VI-E2	Flash 使用分析	17
III-D1	全局五分类网络	6	VI-E3	RAM 使用分析	18
III-D2	pinch/clench 二分类网络	8	VI-E4	模型参数量与存储开销 .	18
III-D3	短时序列校验网络	8	VI-F	性能评估总结	18
III-D4	质心距离模板校验	10	VI-G	赛题需求符合性分析	18
III-E	决策状态机	10	VI-G1	嵌入式功能需求	18
III-E1	小手势与大手势的分类 .	10	VI-G2	ISR 约束	18
III-E2	小手势聚合机制	10	VI-G3	BLE GATT 服务需求 .	18
III-E3	大手势连续确认机制 . .	10	VI-G4	加分项	18
III-E4	不应期控制	11			
III-E5	空闲锁定机制	11	VII	团队介绍及分工	19
III-F	算法完整推理流程	11	VII-A	团队成员	19
IV	BLE GATT 服务实现	12	VIII	结论与展望	20
IV-A	BLE 服务架构设计	12	参考文献		20
IV-B	UUID 设计	12			
IV-C	Characteristic 属性配置	12			
IV-D	数据格式	12			
IV-E	服务实现细节	13			
IV-F	BLE 工程构建	13			
V	嵌入式软件工作流程	13			
V-A	系统启动流程	13			
V-B	任务架构	14			
	V-B1 imu_task (IMU 数据采	14			
	集任务)	14			
	V-B2 algo_task (手势识别任	14			
	务)	14			
V-C	数据流与 FIFO 机制	14			
V-D	CRC-8/SMBus 校验	15			
V-E	VPI 事件系统	15			
V-F	IMU 硬件配置	15			
V-G	输出格式	15			
VI	验证及测试结果	16			
VI-A	验证方法概述	16			
VI-B	QEMU 测试序列	16			
VI-C	QEMU 验证结果	16			
	VI-C1 CRC-8/SMBus 校验 . .	16			
	VI-C2 手势识别精度	16			

II. 方案介绍

A. 系统总体架构

本方案的整体架构如图 1 所示, 采用分层设计。系统自下而上包括硬件抽象层、数据采集层、特征提取层、分类推理层、决策逻辑层和通信服务层。

系统运行在 RISC-V nuclei_evalsoc 处理器上, 使用 FreeRTOS 实时操作系统进行任务调度 [6], [7]。芯原 VPI 事件系统负责模块间消息传递。VPI 的全称为 VeriSilicon Platform Interface, 用于连接中断服务程序与任务、任务与任务之间的异步流程。

核心数据链路如下: IMU 硬件产生数据就绪中断, 服务程序通过 VPI 通知 imu_task; imu_task 经 HAL 接口读取 FIFO 数据、更新 CRC 校验并将样本写入共享 FIFO; 随后 VPI 通知 algo_task, 由算法任务完成滑动窗口管理、特征提取、分类推理和状态机决策; 检测到手势后, 系统通过 UART 输出结果并更新 BLE 服务。整条链路可在一个 IMU 采样周期内完成, 采样周期为 20 ms。

B. 平台规格与约束

目标平台的关键参数如表 I 所示。

在上述硬件约束下, 我们的算法需要满足以下设计要求:

II. 方案介绍

A. 系统总体架构

本方案的整体架构如图 1 所示, 采用分层设计。系统自下而上包括硬件抽象层、数据采集层、特征提取层、分类推理层、决策逻辑层和通信服务层。

系统运行在 RISC-V nuclei_evalsoc 处理器上, 使用 FreeRTOS 实时操作系统进行任务调度 [6], [7]。芯原 VPI 事件系统负责模块间消息传递。VPI 的全称为 VeriSilicon Platform Interface, 用于连接中断服务程序与任务、任务与任务之间的异步流程。

核心数据链路如下：IMU 硬件产生数据就绪中断，服务程序通过 VPI 通知 `imu_task`；`imu_task` 经 HAL 接口读取 FIFO 数据、更新 CRC 校验并将样本写入共享 FIFO；随后 VPI 通知 `algo_task`，由算法任务完成滑动窗口管理、特征提取、分类推理和状态机决策；检测到手势后，系统通过 UART 输出结果并更新 BLE 服务。整条链路可在一个 IMU 采样周期内完成，采样周期为 20 ms。

B. 平台规格与约束

目标平台的关键参数如表 I 所示。

在上述硬件约束下，我们的算法需要满足以下设计要求：

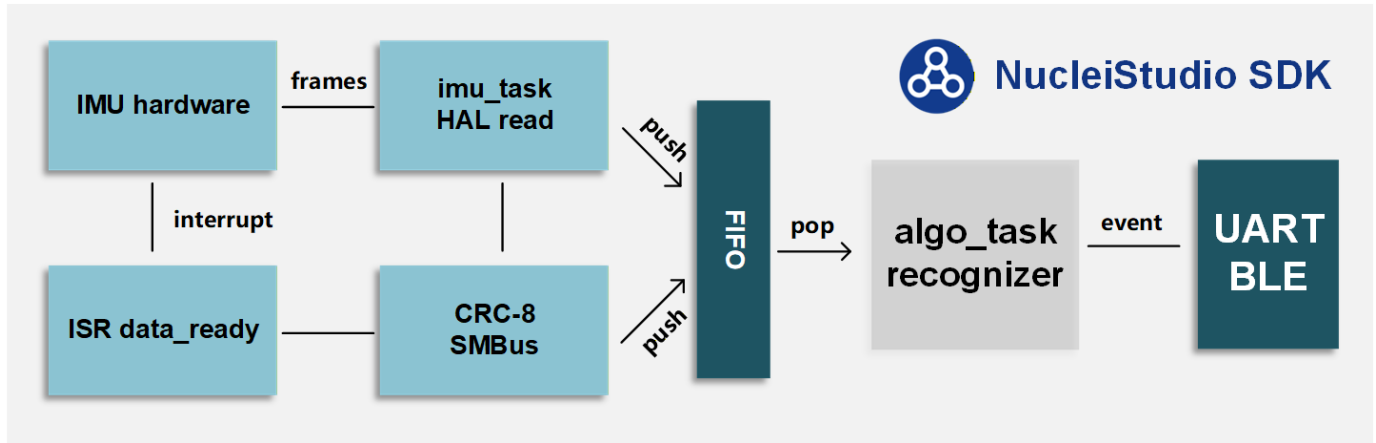


图 1: 系统软件架构总览。IMU 硬件通过中断触发 imu_task 进行数据读取和 CRC 计算，样本通过 FIFO 传递至 algo_task 进行手势识别，识别结果通过 UART 输出并可由 BLE 服务推送。

表 I: 目标平台硬件规格

参数	规格
处理器架构	RISC-V nuclei_evalsoc
Flash 容量	3 MB
RAM 容量	256 KB
操作系统	FreeRTOS
IMU 采样率	50 Hz, 6 轴同步采集
陀螺仪量程	±2000 dps
加速度计量程	±8 g
通信接口	BLE + UART

- 模型参数以 `const float` 数组形式存储于 Flash 中，避免占用 RAM。
- 推理过程不使用动态内存分配，所有缓冲区在编译时静态确定。
- 滑动窗口处理使用环形缓冲区，避免大量数据拷贝。
- 单次推理延迟不超过一个采样周期，即 20 ms。

代码 1 给出了手势识别器的核心数据结构。该结构体包含滑动窗口环形缓冲区、滤波器状态、分类置信度历史和状态机运行时变量，作为静态全局变量分配，无动态内存开销。

```

1 typedef struct GestureRecognizer_st {
2     int16_t ring[GESTURE_WINDOW_SIZE][GESTURE_AXIS_COUNT];
3     float filter_ring[GESTURE_WINDOW_SIZE]
4         [GESTURE_FILTER_FEATURE_COUNT];
5     float filter_low[GESTURE_AXIS_COUNT];
6     float filter_high[GESTURE_AXIS_COUNT];
7     float filter_mid[GESTURE_AXIS_COUNT];
8     float filter_prev[GESTURE_AXIS_COUNT];
9     float score_history[8][7];
10    uint16_t write_pos;
11    uint16_t sample_count;
12    uint16_t stride_count;

```

```

13    uint32_t total_samples;
14    uint8_t score_history_pos;
15    uint8_t score_history_count;
16    bool filter_initialized;
17    GestureLabel run_label;
18    uint8_t run_count;
19    uint8_t small_count;
20    bool small_wait_idle;
21    uint8_t small_idle_count;
22    float small_pinch_sum;
23    float small_clench_sum;
24    float small_temporal_pinch_sum;
25    float small_temporal_clench_sum;
26    uint32_t last_emit_ms;
27 } GestureRecognizer;

```

Listing 1: 手势识别器核心数据结构 (gesture_algorithm.h)

C. 数据集概况

竞赛提供了 VeriHealthi IMU Dataset，数据集的统计信息如表 II 和图 2 所示。

表 II: VeriHealthi IMU 数据集统计

手势类别	录制数量	采集者数量	含标签
pinch, 双指互点	154	6	是
clench, 握拳	154	6	是
up, 抬腕	150	6	是
down, 放下	150	6	是
others, 其他	57	6	否
合计	665	6	—

如图 2 中 a 和 b 所示，数据集共包含 665 段 IMU 录制，覆盖 pinch、clench、up、down 和 others 五类动

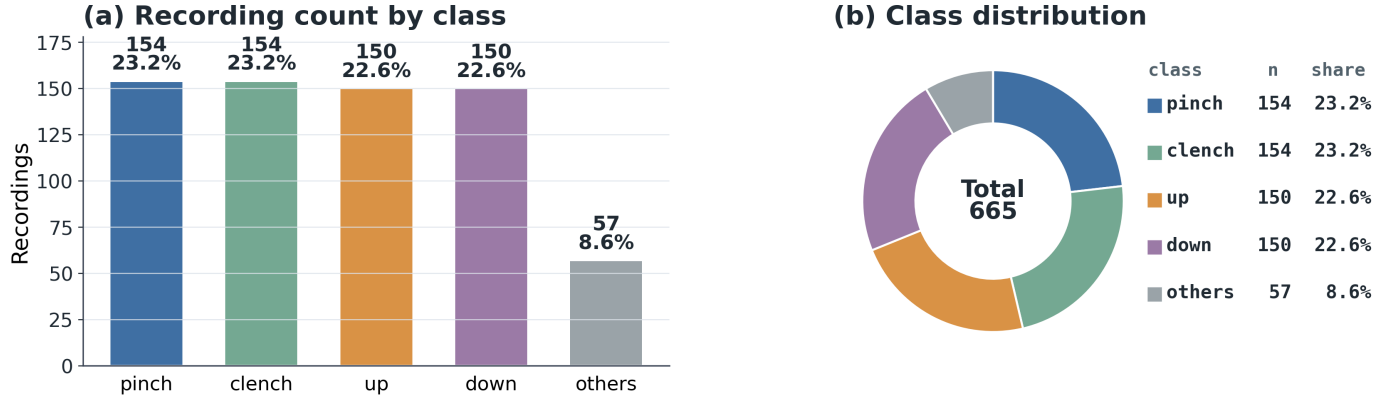


图 2: 数据集各类别录制数量分布及占比。四类手势数据相对均衡, others 类用于训练负类样本。

作。其中四类目标手势的样本规模较为接近: pinch 和 clenched 各 154 段, 分别占 23.2%; up 和 down 各 150 段, 分别占 22.6%。这种分布使模型在四类目标手势上能够获得相对均衡的训练样本, 减少由类别数量差异带来的偏置。others 类共有 57 段, 占 8.6%, 主要用于表示非目标手势和背景干扰动作, 在训练中承担负样本作用, 帮助模型降低误触率。

数据集中每条录制包含一个 IMU 原始数据文件和一个对应的 CSV 标签文件。数据来自 6 位采集者, 编号为 ID0-ID5, 涵盖左手和右手录制。up 和 down 共享同一类录制文件, 文件名以 IMU_updown_* 开头, 同一段录制中交替包含两种手势事件, 需要依据标签文件区分。

数据文件的前 5 行为元信息头, 从第 6 行开始为 7 列数值数据, 其中第 1 列为序列号, 第 2-4 列为陀螺仪原始值, 第 5-7 列为加速度计原始值。

D. 评估指标

竞赛采用基于事件窗口匹配的 F1 评估方法。对于每个标注事件, 系统需要在事件发生时间前 0.3 秒到事件发生时间之间输出正确类别, 才记为真正例。真正例、假正例和假负例分别记为 TP、FP 和 FN。评估指标包括:

- **Precision:** 精确率, $P = TP / (TP + FP)$
- **Recall:** 召回率, $R = TP / (TP + FN)$
- **F1 Score:** F1 分数, $F_1 = 2PR / (P + R)$
- **Macro F1:** 宏平均 F1, 四类手势 F1 的算术平均

此外, 系统需要在 QEMU 回放开始后输出 CRC-8/SMBus 校验值。该值对 IMU 原始数据前 320000 字节计算, 用于验证数据读取顺序和字节处理逻辑。竞赛

评估采用固定的 QEMU 测试序列, 详见第 VI 节。该序列由多位用户的多段录制拼接而成, 总计 87 个标注手势事件。

III. 算法设计

本节介绍数据预处理、特征工程、分类推理和状态机决策逻辑。算法先用全局网络给出候选类别, 再对容易混淆的小手势进行二分类和时序校验, 最后由状态机输出事件。

A. 数据预处理与滑动窗口

1) 原始数据格式转换: IMU 硬件返回的原始数据为 16 位有符号整数 (int16_t), 需要转换为物理单位。陀螺仪数据的转换系数为 16.4 LSB/dps, 加速度计的转换系数为 4096 LSB/g。在嵌入式实现中, 为了节省 RAM 和计算开销, 统计特征的提取直接使用原始整数值, 仅在滤波器组计算时进行物理单位转换。这一设计减少了浮点运算次数, 同时保证了特征值的数值稳定性。

2) 滑动窗口机制: 系统采用固定长度的滑动窗口对连续的 IMU 数据流进行分帧处理, 如图 3 所示。

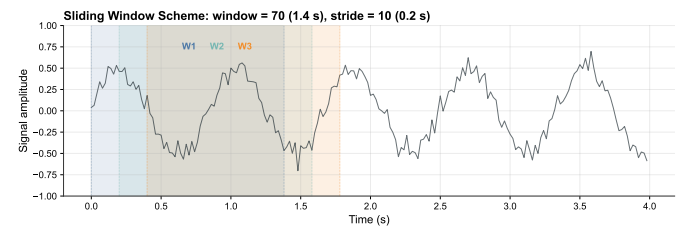


图 3: 滑动窗口分帧示意图。窗口大小为 70 个采样点, 对应 1.4 秒; 步长为 10 个采样点, 对应 0.2 秒; 相邻窗口有 85.7% 的重叠。

窗口参数的设计依据如下：

- 窗口大小 $W = 70$ ：对应 1.4 秒数据。该长度覆盖了 pinch、clench、up 和 down 的典型持续时间，同时保留足够的统计稳定性。实验比较了 50–100 个采样点的窗口，70 个采样点在检测精度和响应延迟之间较为均衡。
- 步长 $S = 10$ ：对应 0.2 秒数据。系统每 0.2 秒产生一次推理输出，状态机可以得到足够密集的决策信息。相邻窗口之间有 $60/70 = 85.7\%$ 的重叠，有利于平滑时间维度上的类别变化。

在嵌入式实现中，滑动窗口通过环形缓冲区管理。GestureRecognizer 结构体中包含 70×6 的 `int16_t` 数组 `ring`，用于存储原始 IMU 数据；另有 70×4 的 `float` 数组 `filter_ring`，用于存储滤波器特征。写入位置由 `write_pos` 追踪，每收到一个新样本后递增并取模。当 `sample_count` 达到窗口大小 W 后，每隔 S 个样本触发一次特征提取与推理。

环形缓冲区的使用避免了每次窗口滑动时对整个窗口数据的拷贝操作。特征提取时，通过合理的索引计算，从环形缓冲区中直接读取当前窗口对应的数据，实现了 $O(1)$ 的窗口滑动开销。

B. 特征工程

本方案使用 85 维特征向量，由 77 维统计特征和 8 维滤波器组特征组成。前者提供了可解释的时域统计描述，后者引入了频域分量的判别信息。

1) 77 维统计特征：一个滑动窗口包含 6 条同步信号，分别为三轴陀螺仪 g_x 、 g_y 、 g_z 和三轴加速度计 a_x 、 a_y 、 a_z 。统计特征的设计从单轴幅度开始：均值、标准差、峰值、能量和过零率描述每条轴在窗口内的基本变化。随后加入轴间相关系数，用来刻画手腕旋转时多轴之间的协同变化。对于 up 和 down 这类大幅度动作，单轴统计量容易受佩戴方向影响，因此进一步计算合加速度和合角速度。对于 pinch 和 clench 这类小手势，仅看幅度不足以稳定区分，还需要加入加速度与角速度之间的相对强弱以及分布宽度。最终得到表 III 中的 77 维统计特征。

这里的 x_i 表示某一通道在窗口内的第 i 个采样点， $N = 70$ 为窗口长度。 a_i 和 g_i 在比值特征中表示同一坐标轴上的加速度和角速度幅值， $i \in \{x, y, z\}$ ； ϵ 是很小的正数，用于避免角速度接近零时出现除零。四分位距 IQR 的英文全称为 Interquartile Range，其中 Q_{25}

表 III: 77 维统计特征构成

特征类型	维度	说明
均值 mean	6	各轴信号均值
标准差 std	6	各轴信号波动程度
最大值 max	6	各轴信号峰值
最小值 min	6	各轴信号谷值
峰峰值 range	6	$\max - \min$ ，信号动态范围
能量 energy	6	$\sum x_i^2 / N$ ，信号强度
过零率 ZCR	6	信号穿越零点的频率
绝对均值 abs mean	6	$\sum x_i / N$
轴间相关 corr	15	$\binom{6}{2}$ 个轴对的 Pearson 相关系数
合加速度均值	1	$\sqrt{a_x^2 + a_y^2 + a_z^2}$
合加速度标准差	1	$\text{std}(\sqrt{a_x^2 + a_y^2 + a_z^2})$
合角速度均值	1	$\sqrt{g_x^2 + g_y^2 + g_z^2}$
合角速度标准差	1	$\text{std}(\sqrt{g_x^2 + g_y^2 + g_z^2})$
加速度与陀螺仪比值	6	三轴 $r_i = a_i / (g_i + \epsilon)$ 的均值和最大值
四分位距 IQR	4	$Q_{75} - Q_{25}$ ，描述中间 50% 样本跨度
合计	77	—

为第 25 百分位数， Q_{75} 为第 75 百分位数。IQR 只关注排序后中间一半样本的跨度，比最大值和最小值更不容易受孤立尖峰影响。

嵌入式实现中，统计特征采用单遍扫描计算。遍历窗口数据时同步维护运行和与运行平方和，可同时得到均值、标准差和能量等统计量，避免多次遍历窗口。四分位距 IQR 使用选择算法直接寻找第 25 和第 75 百分位数，时间复杂度为 $O(N)$ 。

2) 8 维滤波器组特征：参考文献 [1] 中的信号分解方法，本方案设计了一组单极点 IIR 滤波器，将 IMU 信号分解为低频趋势、高频分量和中频手势分量。中频分量约为 8–20 Hz，与手指和手腕运动的主要频段相近，携带较多判别信息。

记 $\mathbf{x}[n] \in \mathbb{R}^6$ 为第 n 个六轴 IMU 采样点，三个陀螺仪轴和三个加速度计轴按同一递推公式逐元素计算。 $\mathbf{l}[n]$ 、 $\mathbf{h}[n]$ 和 $\mathbf{m}[n]$ 分别表示低通输出、高通输出和中频输出，它们都是 6 维向量； $\mathbf{x}[n-1]$ 为上一采样点， $\mathbf{l}[n-1]$ 、 $\mathbf{h}[n-1]$ 、 $\mathbf{m}[n-1]$ 为上一时刻的滤波器状态。滤波器组的递推形式如下：

低通滤波器，等效截止频率 7.99 Hz：

$$\mathbf{l}[n] = \mathbf{l}[n-1] + \alpha_{\text{LP}} \cdot (\mathbf{x}[n] - \mathbf{l}[n-1]) \quad (1)$$

其中 $\alpha_{\text{LP}} = 0.5010$ 为低通滤波系数。该式等价于指数滑动平均：当前输入 $\mathbf{x}[n]$ 与上一低通状态 $\mathbf{l}[n-1]$ 的差值先乘以 α_{LP} ，再累加到上一状态上。在 $f_s = 50$ Hz

时对应 $f_c = 7.9896$ Hz。该分支用于保留手臂姿态和慢速握拳带来的低频趋势。

高通滤波器，等效截止频率 8.02 Hz:

$$\mathbf{h}[n] = \alpha_{\text{HP}} \cdot (\mathbf{h}[n-1] + \mathbf{x}[n] - \mathbf{x}[n-1]) \quad (2)$$

其中 $\alpha_{\text{HP}} = 0.4980$ 为高通滤波系数。括号内的 $\mathbf{x}[n] - \mathbf{x}[n-1]$ 描述相邻采样点的变化量，上一高通状态 $\mathbf{h}[n-1]$ 用于保持递推连续性。按照一阶 RC 离散化关系 $f_c = f_s(1 - \alpha)/[2\pi\alpha]$ 反推，在 $f_s = 50$ Hz 时对应 $f_c = 8.0217$ Hz。该滤波器会削弱缓慢变化的姿态趋势，保留短时间内快速变化的部分，适合捕捉 pinch 触碰瞬间产生的细小振动。

中频带通滤波器，通带约 8.02–20.00 Hz:

$$\mathbf{m}[n] = \mathbf{m}[n-1] + \alpha_{\text{BP}} \cdot (\mathbf{h}[n] - \mathbf{m}[n-1]) \quad (3)$$

其中 $\alpha_{\text{BP}} = 0.7154$ 为带通低通系数。对高通输出 $\mathbf{h}[n]$ 再做一次低通平滑；按同一系数关系反推，该级低通等效截止频率为 20.0034 Hz。经过前级高通和本级低通后，剩余部分主要落在 8.02–20.00 Hz 附近，因此记为中频分量 $\mathbf{m}[n]$ 。

从每个时刻的滤波器输出中提取 4 个标量特征：

- 1) $\|\mathbf{m}_{\text{gyro}}\|_2$: 中频陀螺仪能量——反映手指/手腕微小运动产生的中频旋转信号
- 2) $\|\mathbf{m}_{\text{accel}}\|_2$: 中频加速度计能量——反映手指/手腕微小运动产生的中频线性加速度
- 3) $\|\mathbf{l}_{\text{gyro}}\|_2$: 低通陀螺仪能量——反映手臂的缓慢旋转趋势
- 4) $|\|\mathbf{l}_{\text{accel}}\|_2 - 1|$: 低通加速度计去重力偏差，用于反映手臂姿态偏移

对滑动窗口内的 4 个滤波器特征序列分别计算均值和最大值，得到 $4 \times 2 = 8$ 维滤波器组特征。pinch 主要表现为手指微小运动产生的中高频振动；up 和 down 伴随手臂大幅运动；clench 则更多体现为握拳时的慢速肌肉收缩。频段分解有助于区分这三类运动形态。

图 4 展示了滤波器组在 pinch 录制上的分解效果。手势发生时段的中频能量升高，低通去趋势分量则对应手臂姿态的缓慢变化。

滤波器组采用流式计算模式。每收到一个新的 IMU 样本，系统更新低通、高通、中频和上一样本状态，并将当前时刻的 4 个标量特征写入 `filter_ring`。该实现避免了在特征提取阶段重新遍历窗口数据，滤波器更新开销为 $O(1)$ 。

C. 原始 IMU 信号分析

频域特征和状态机策略的设计来自原始 IMU 信号的差异，图 5–图 8 给出了典型录制的波形和幅度对比。从幅度和频率两方面看，pinch 和 clench 都属于小手势，但二者仍存在差别：pinch 更接近短促振动，clench 更接近中等幅度的慢速变化。up 和 down 则伴随前臂旋转，陀螺仪和加速度计幅度均明显更大。others 类没有稳定形态，包含走路、摆臂和日常手部活动。

综合原始波形可以看到，四类动作不仅幅度大小不同，频率范围、持续时间和事件间隔也存在差异。pinch 的主要线索是短促高频分量，clench 更依赖中低频幅度变化，up/down 则表现为连续的大幅旋转。后续分类框架据此采用“窗口特征判别 + 短时序列校验 + 状态机聚合”的结构：窗口特征负责给出当前片段的候选类别，时序校验检查多个窗口的一致性，状态机再把连续窗口转化为最终事件。

D. 分类推理框架

特征提取完成后，算法并不直接用单个窗口给出事件，而是先得到几个互补的判别量，再交给状态机做事件级输出。全局五分类网络负责回答“当前窗口最像哪一类”，pinch/clench 二分类网络负责回答“小手势内部更像双指互点还是握拳”，短时序列校验负责判断最近多个窗口是否在时间上连续，质心距离校验负责排除偏离训练样本分布的异常小手势。推理流程如图 9 所示。

1) 全局五分类网络：下面先给出统一符号。 $\mathbf{f} \in \mathbb{R}^{85}$ 表示标准化后的 85 维特征向量； $\mathbf{W}$ 和 $\mathbf{b}$ 分别表示神经网络的权重矩阵和偏置向量，训练完成后以 `const float` 数组固化到 Flash。 $\text{ReLU}(u) = \max(u, 0)$ 为激活函数。网络输出记为 `logit`，`logit` 是 softmax 之前的原始类别分数，数值越大表示该类越可能被选中。全局网络输出 $\mathbf{z}_{\text{NN}}$ 的 5 个分量按 [none, pinch, clench, up, down] 排列，其中 NN 表示 Neural Network；二分类网络输出 $\mathbf{z}_{\text{PC}}$ 的 2 个分量按 [pinch, clench] 排列，其中 PC 表示 pinch/clench。

全局五分类网络是主干分类器，对 none、pinch、clench、up 和 down 进行初步判别。网络结构为三层多层感知机，英文为 Multi-Layer Perceptron，简称 MLP：

$$\begin{aligned} \mathbf{h}_1 &= \text{ReLU}(\mathbf{W}_1 \mathbf{f} + \mathbf{b}_1), \quad \mathbf{h}_1 \in \mathbb{R}^{64} \\ \mathbf{h}_2 &= \text{ReLU}(\mathbf{W}_2 \mathbf{h}_1 + \mathbf{b}_2), \quad \mathbf{h}_2 \in \mathbb{R}^{32} \\ \mathbf{z}_{\text{NN}} &= \mathbf{W}_3 \mathbf{h}_2 + \mathbf{b}_3, \quad \mathbf{z}_{\text{NN}} \in \mathbb{R}^5 \end{aligned} \quad (4)$$

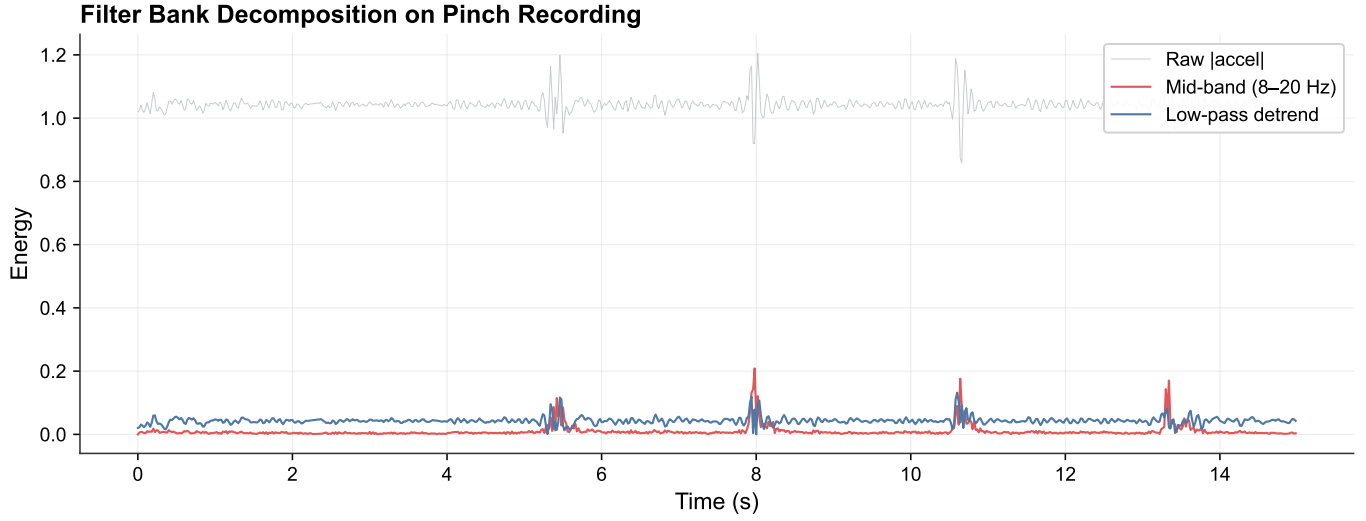


图 4: 滤波器组在 pinch 录制上的分解效果。中频分量约为 8–20 Hz，能够反映手指微小运动产生的周期性信号模式。

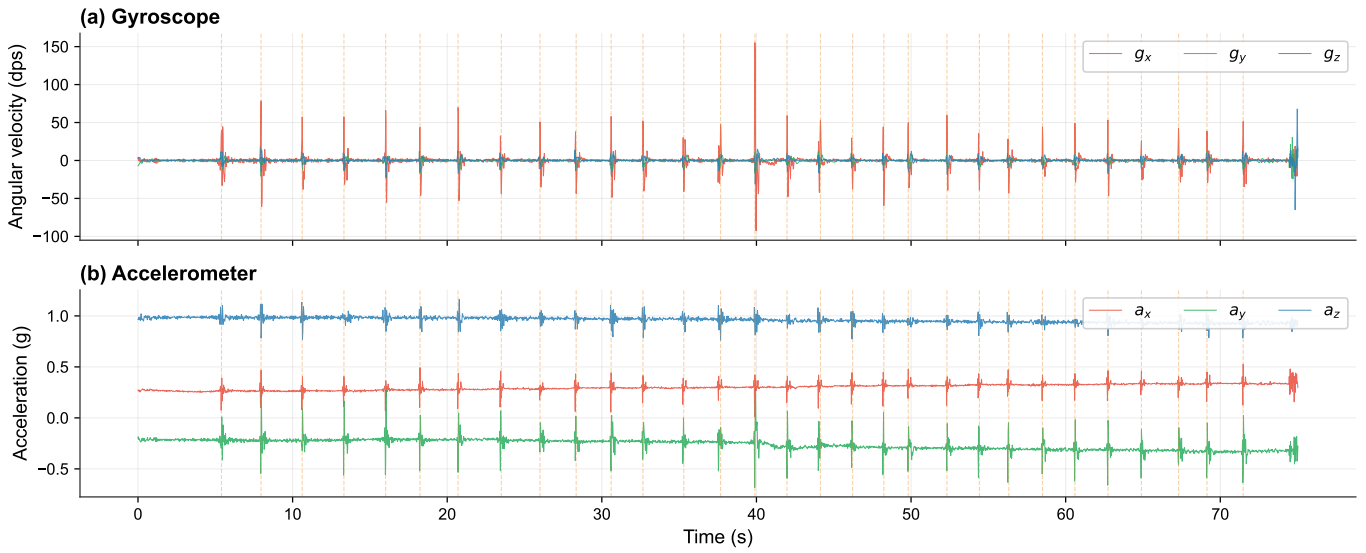


图 5: Pinch 录制的原始 IMU 6 轴波形。上：陀螺仪三轴角速度；下：加速度计三轴加速度。橙色虚线为标注的手势事件时间点。

式中 $\mathbf{h}_1$ 和 $\mathbf{h}_2$ 是隐藏层特征，维度分别为 64 和 32； $\mathbf{z}_{\text{NN}} \in \mathbb{R}^5$ 是五分类 logit 向量。对应的矩阵维度为 $\mathbf{W}_1 \in \mathbb{R}^{64 \times 85}$ 、 $\mathbf{W}_2 \in \mathbb{R}^{32 \times 64}$ 、 $\mathbf{W}_3 \in \mathbb{R}^{5 \times 32}$ 。推理时先对原始 85 维特征做 z-score 标准化：

$$\mathbf{f} = (\mathbf{f}_{\text{raw}} - \boldsymbol{\mu}) \odot \boldsymbol{\sigma}^{-1} \quad (5)$$

其中 $\mathbf{f}_{\text{raw}}$ 为未标准化特征， $\boldsymbol{\mu}$ 为训练集均值， $\boldsymbol{\sigma}^{-1}$ 为标准差倒数， $\odot$ 表示逐元素乘法。标准化参数在训练阶段计算并固化于 Flash 中。

全局网络的候选类别由 $\arg \max_k z_{\text{NN},k}$ 给出。若最

大分量对应 up 或 down，状态机会进入大手势连续确认路径；若对应 pinch 或 clench，则进入小手势聚合路径，后续还需要二分类网络、时序校验和模板校验共同参与判断。

训练细节如下：

- **优化器**：Adam，学习率 10^{-3} ，权重衰减 10^{-4}
- **损失函数**：带类别权重的交叉熵损失，权重按类别频率的倒数计算并归一化
- **训练数据构建**：使用全部录制的特征数据，负类 none 每 3 个窗口保留 1 个样本以缓解类别不平衡

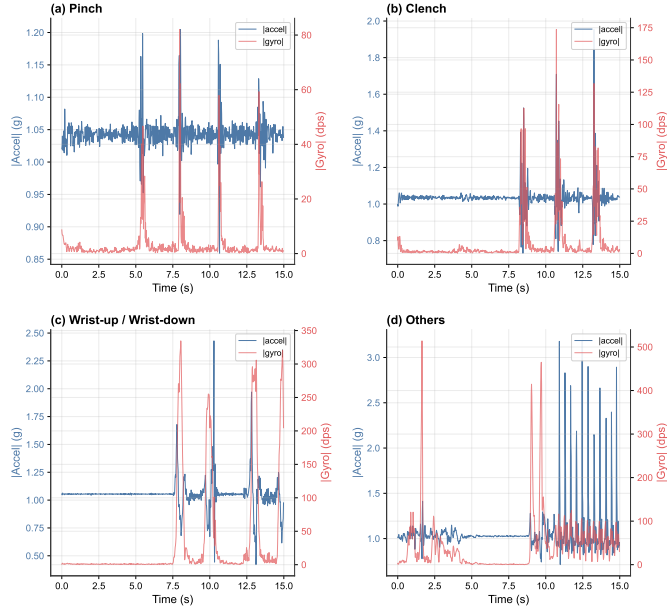


图 6: 四类手势的 IMU 信号幅度对比。pinch: 微弱高频振动; clench: 中等幅度慢速变化; up/down: 大幅度手臂旋转信号; others: 随机运动模式。

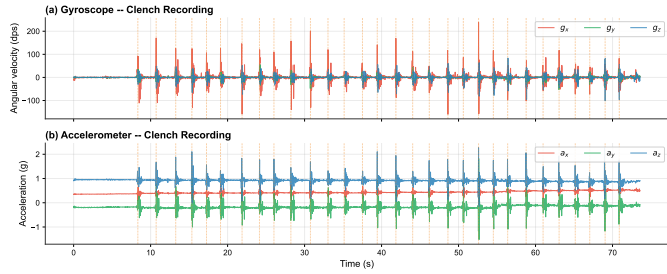


图 7: Clench 录制的原始 IMU 6 轴波形。握拳动作在加速度计上产生中等幅度的信号变化，陀螺仪响应相对较弱。

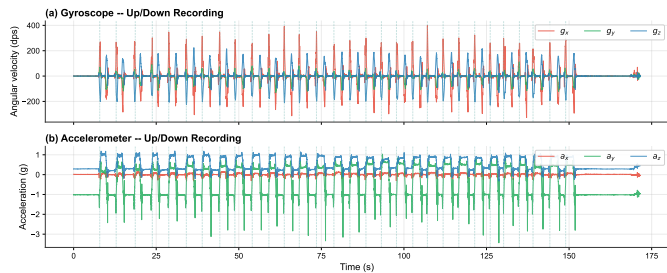


图 8: Up/down 录制的原始 IMU 6 轴波形。手臂旋转动作在陀螺仪和加速度计上均产生大幅度的特征信号。

- 训练轮数: 60 个 epoch, batch size 为 512
- 随机种子: 固定为 20260607, 保证训练过程可复现

全局分类网络的参数量为 $85 \times 64 + 64 + 64 \times 32 +$

$32 + 32 \times 5 + 5 = 7,781$ 个浮点数, 占用 Flash 空间约 31.1 KB。网络隐藏层使用 ReLU 激活函数, 输出层直接给出 5 类 logit 值。本文采用 MLP 而非卷积神经网络或循环神经网络, 原因是 85 维输入已经包含统计和频域信息, MLP 可以完成特征到类别的映射, 同时推理仅涉及矩阵乘法和逐元素运算, 便于在带单精度 FPU 的 RISC-V 平台上运行。

2) *pinch/clench* 二分类网络: *pinch* 和 *clench* 在幅度上都属于小手势, 单靠全局五分类网络容易在二者之间产生混淆。因此, 算法对小手势候选再运行一个二分类网络。该网络的输入仍是同一个 $\mathbf{f}$, 但训练样本只包含 *pinch* 和 *clench* 两类:

$$\begin{aligned} \mathbf{h}_1^{\text{PC}} &= \text{ReLU}(\mathbf{W}_1^{\text{PC}}\mathbf{f} + \mathbf{b}_1^{\text{PC}}), \quad \mathbf{h}_1^{\text{PC}} \in \mathbb{R}^{32} \\ \mathbf{h}_2^{\text{PC}} &= \text{ReLU}(\mathbf{W}_2^{\text{PC}}\mathbf{h}_1^{\text{PC}} + \mathbf{b}_2^{\text{PC}}), \quad \mathbf{h}_2^{\text{PC}} \in \mathbb{R}^{16} \\ \mathbf{z}_{\text{PC}} &= \mathbf{W}_3^{\text{PC}}\mathbf{h}_2^{\text{PC}} + \mathbf{b}_3^{\text{PC}}, \quad \mathbf{z}_{\text{PC}} \in \mathbb{R}^2 \end{aligned} \quad (6)$$

上式中, 上标 PC 表示这组参数属于 *pinch/clench* 二分类网络。 $\mathbf{h}_1^{\text{PC}}$ 和 $\mathbf{h}_2^{\text{PC}}$ 分别为 32 维和 16 维隐藏层, $\mathbf{z}_{\text{PC}} \in \mathbb{R}^2$ 为二分类 logit 向量。 $z_{\text{PC},0}$ 对应 *pinch*, $z_{\text{PC},1}$ 对应 *clench*。该网络只在全局网络给出小手势候选时参与决策; 若全局网络输出 up、down 或 none, 二分类结果不会改变当前窗口类别。

二分类网络采用 32-16-2 结构, 小于全局网络的 64-32-5 结构。二分类任务的边界更集中, 不需要同等规模的模型容量。该网络独立训练, 输入仍为 85 维特征, 训练样本仅来自 *pinch* 和 *clench* 两类。

二分类网络的参数量为 $85 \times 32 + 32 + 32 \times 16 + 16 + 16 \times 2 + 2 = 3,378$ 个浮点数, 占用 Flash 约 13.5 KB。

为了补偿公开训练集中 *clench* 被误分为 *pinch* 的错误模式, 推理时对 *clench* 对应的二分类 logit 加入一个固定偏置 $\beta_{\text{clench}} = 4.0$:

$$z'_{\text{PC},\text{clench}} = z_{\text{PC},\text{clench}} + \beta_{\text{clench}} \quad (7)$$

偏置只作用在二分类网络的 *clench* 分量上, 不改变全局五分类网络对 up 和 down 的判断。小手势聚合结束时, 状态机会比较累计后的 *pinch* 得分和 *clench* 得分, 偏置后的 $z'_{\text{PC},\text{clench}}$ 参与 *clench* 累计得分。

3) 短序列校验网络: 单帧分类容易受到瞬态噪声影响, 尤其在手势起始和结束阶段, 局部特征可能接近静止状态或其他手势。为此, 本方案使用一个轻量级时序卷积网络对最近 8 个窗口的 logit 序列进行建

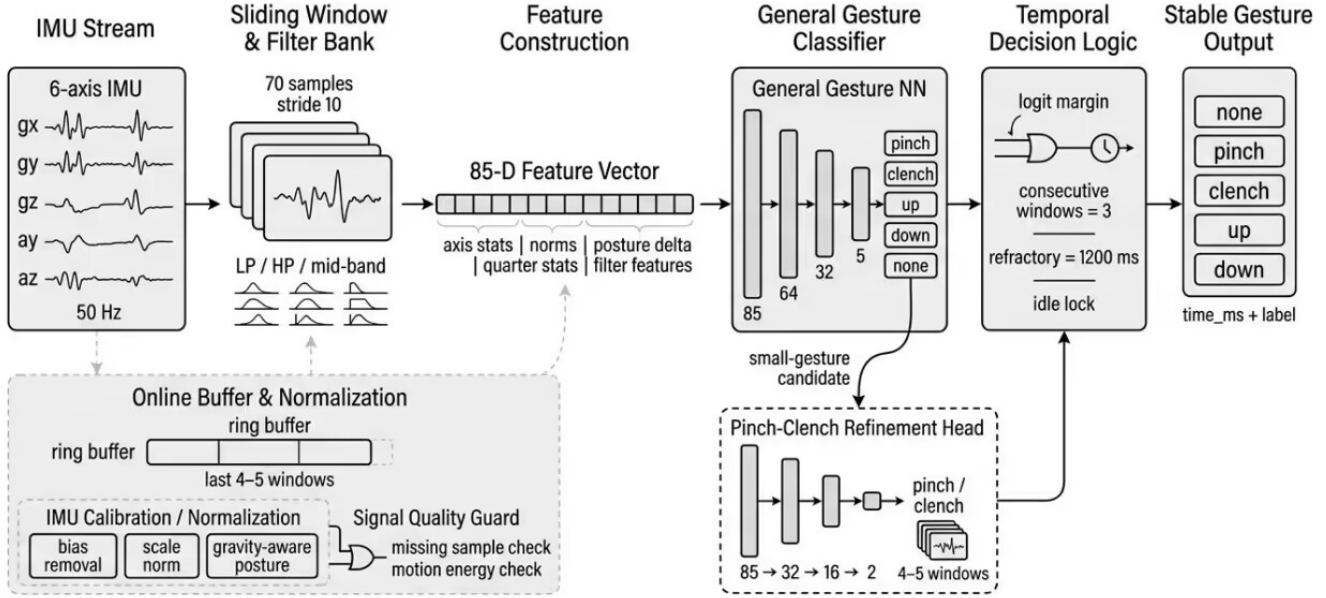


图 9: 分类推理流程图。85 维特征首先进入全局五分类网络和 pinch/clench 二分类网络，随后由短时序列校验和模板距离校验补充判别信息，最终由状态机输出手势事件。

模 [4]。时序卷积网络的英文为 Temporal Convolutional Network，简称 TCN。这里的 TCN 只做后处理校验，不涉及强化学习。

$$\begin{aligned} \mathbf{s}[t] &= [\mathbf{z}_{\text{NN}}[t]; \mathbf{z}_{\text{PC}}[t]] \in \mathbb{R}^7 \\ \mathbf{S} &= [\mathbf{s}[t-7], \dots, \mathbf{s}[t]] \in \mathbb{R}^{8 \times 7} \end{aligned} \quad (8)$$

其中 t 表示当前推理窗口的序号， $\mathbf{s}[t]$ 把当前窗口的 5 维全局 logit 和 2 维二分类 logit 拼接为 7 维向量。 $\mathbf{S}$ 为最近 8 个窗口的 logit 历史，第一维是时间，第二维是类别分数。8 个窗口对应约 1.6 秒历史，可为状态机提供时间一致性信息。

TCN 采用两层因果一维卷积，核大小 $k = 3$ ，隐藏维度为 8。因果卷积只使用当前窗口和过去窗口，不使用未来信息，因此可以在线运行：

$$\begin{aligned} \mathbf{H}_1 &= \text{ReLU}(\text{CausalConv1d}(\mathbf{S}, \mathbf{W}_{c1})) \in \mathbb{R}^{8 \times 8} \\ \mathbf{H}_2 &= \text{ReLU}(\text{CausalConv1d}(\mathbf{H}_1, \mathbf{W}_{c2})) \in \mathbb{R}^{8 \times 8} \\ \mathbf{u}_{\text{TCN}} &= \mathbf{H}_2[8, :] \in \mathbb{R}^8 \\ \mathbf{z}_{\text{TCN}} &= \mathbf{W}_{\text{fc}} \mathbf{u}_{\text{TCN}} + \mathbf{b}_{\text{fc}} \in \mathbb{R}^5 \end{aligned} \quad (9)$$

式中 $\mathbf{W}_{c1}$ 和 $\mathbf{W}_{c2}$ 为两层卷积核参数， $\mathbf{H}_1$ 和 $\mathbf{H}_2$ 为

每个时间位置上的 8 维隐藏表示。 $\mathbf{u}_{\text{TCN}}$ 取最后一个时间位置的隐藏向量，即当前窗口在过去 7 个窗口背景下的时序表示； $\mathbf{W}_{\text{fc}}$ 和 $\mathbf{b}_{\text{fc}}$ 把该表示映射为 5 维时序校验 logit。 $\mathbf{z}_{\text{TCN}}$ 的类别顺序与 $\mathbf{z}_{\text{NN}}$ 一致。

短时序列校验主要用于抑制孤立的假脉冲：

- 当全局网络在单个窗口突然输出高置信度手势预测，而相邻窗口均为 none 时，时序校验会降低该手势的有效性。
- 当全局网络在多个连续窗口输出一致类别时，时序校验会增强该类别在后续状态机中的权重。

时序校验网络训练时采用两类数据增强：

- 高斯噪声注入：以标准差 0.04 和 0.08 两个级别向输入 score 序列添加随机噪声，增强模型对 logit 值波动的鲁棒性
- 随机缩放：以 0.92–1.08 倍的随机因子对输入序列进行缩放，模拟不同置信度水平下的输入变化

经数据增强，训练样本扩展为 4 倍，以缓解小模型在有限训练数据上的过拟合。

训练过程为 24 个 epoch，使用带类别权重的交叉熵损失，随机种子为 20260616。该网络的总参数量为

$(3 \times 7 \times 8 + 8) + (3 \times 8 \times 8 + 8) + (8 \times 5 + 5) = 413$ 个浮点数，占用 Flash 约 1.7 KB。

在嵌入式实现中，系统维护一个 8×7 的 `score_history` 环形缓冲区。每次新的全局网络和二分类网络结果到来后，缓冲区写入 7 个 logit 值，随后直接在该缓冲区上进行因果卷积计算。

4) 质心距离模板校验：神经网络给出的是判别分数，但少数异常窗口可能仍落在错误类别一侧。为此，小手势分支还加入质心距离模板校验。该模块不重新训练复杂模型，而是在训练集上为 pinch 和 clench 建立两个特征中心，推理时检查当前样本是否离对应中心过远。

具体实现步骤如下：

- 1) **特征选择**：从 85 维特征中选取特定的 16 个维度，构成模板特征子向量 $\mathbf{q} \in \mathbb{R}^{16}$ 。选择这些维度是因为它们在 pinch 和 clench 之间具有较强判别力。
- 2) **独立标准化**：对模板特征进行独立的 z-score 标准化，使用模板专用的均值和方差参数。
- 3) **质心计算**：分别计算训练集中 pinch 类和 clench 类样本标准化后的均值向量，作为类别质心 $\mathbf{c}_{\text{pinch}}$ 和 $\mathbf{c}_{\text{clench}}$ 。
- 4) **阈值确定**：对训练集中的每个 pinch 样本计算到 pinch 质心的距离，取 97.5 百分位数作为 pinch 类的接受阈值 τ_{pinch} ；clench 类似。
- 5) **推理时校验**：对候选类别 k 计算距离 $d_k = \|\mathbf{q} - \mathbf{c}_k\|_2^2$ 。若 $d_k > \tau_k$ ，说明当前样本偏离该类训练样本中心，对对应类别 logit 施加惩罚，降低其被选中的概率。

模板校验模块的参数量较小。两个质心向量、标准化均值、方差倒数和接受阈值合计 66 个浮点数，占用 Flash 约 0.3 KB。

模板校验模块与神经网络的判别机制不同：神经网络学习非线性决策边界，模板校验模块则用距离度量做异常检测式校验。两者互补，有助于减少单一模型的盲区。

E. 决策状态机

分类网络逐窗口输出 logit 值，状态机将连续窗口结果转化为离散手势事件，同时抑制重复触发。

1) 小手势与大手势的分类：算法将四类手势按运动幅度划分为两组，两组手势对应不同的信号特征和检测策略：

- **小手势**：pinch 和 clench。二者主要由手部微小运动产生，IMU 信号幅度较小、持续时间较短，单窗口分类更容易受噪声影响，因此需要多窗口聚合。
- **大手势**：up 和 down。二者伴随手臂旋转，信号幅度较大、持续时间较长，单窗口分类置信度较高，连续确认即可降低偶发误判。

2) 小手势聚合机制：对于 pinch 和 clench，系统采用以下聚合策略：

- 1) **进入条件**：当全局网络输出的最大 logit 对应 pinch 或 clench，且 logit margin 超过阈值 $\theta_{\text{margin}} = 2.5$ 时，进入小手势聚合状态。
- 2) **聚合累加**：在聚合状态中，累加二分类网络的 pinch 与 clench logit，并加入时序校验网络对两类的 logit。代码中维护 `small_pinch_sum`、`small_clench_sum`、`small_temporal_pinch_sum` 和 `small_temporal_clench_sum` 四个变量。
- 3) **退出检测**：聚合过程中，若当前窗口不再是小手势候选，则进入等待空闲状态。在该状态下连续检测到 2 个 none 类窗口后，聚合结束。
- 4) **窗口计数确认**：
 - 对 pinch，需要至少 $N_{\text{pinch_first}} = 5$ 个连续小手势窗口才能首次发射，后续为 $N_{\text{pinch}} = 4$ 个窗口。
 - 对 clench，需要至少 $N_{\text{clench}} = 4$ 个窗口。
- 5) **最终判定**：当小手势窗口计数达到阈值时，系统比较 pinch 与 clench 的累计得分。得分由二分类网络输出、clench 偏置和时序校验输出构成。初步类别确定后，再通过质心距离模板校验进行一次异常样本检查。

3) 大手势连续确认机制：对于 up 和 down，采用连续确认策略即可：

- 1) 当全局网络输出的最大 logit 对应 up 或 down 时，将该类别与当前连续标签 `run_label` 比较。
- 2) 若与 `run_label` 相同，连续计数 `run_count` 加一。
- 3) 若不同，重置 `run_label` 为当前类别，`run_count` 置为 1。
- 4) 连续 $N_{\text{run}} = 3$ 个窗口输出相同的 up 或 down 类别时，发射手势事件。

up 和 down 手势的前臂旋转幅度大、持续时间长，IMU 信号与静止状态和小手势差异明显，全局 NN 的单帧分类准确率已较高，连续确认 3 个窗口即可排除偶发误判。

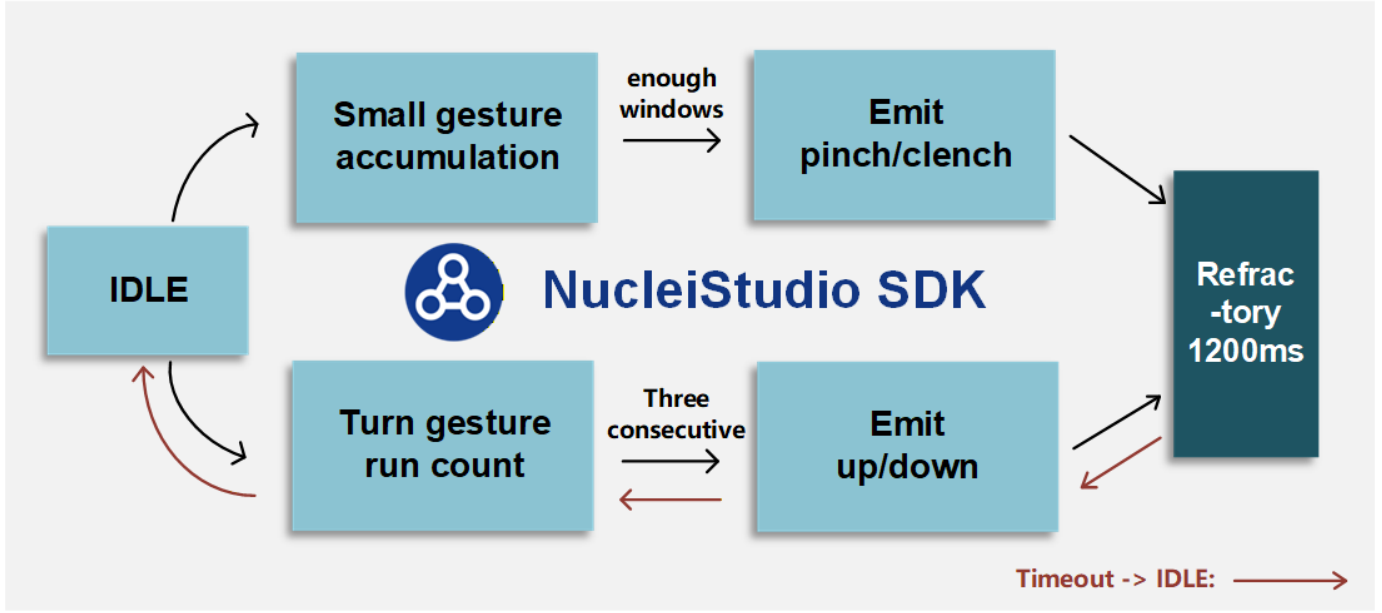


图 10: 手势决策状态机。系统从 IDLE 状态出发，根据手势类型分别进入小手势聚合路径或大手势连续确认路径，事件输出后进入不应期。

4) 不应期控制：为防止同一个手势动作被重复检测，每次成功发射手势事件后，系统进入不应期。不应期长度为 $T_{\text{refractory}} = 1200 \text{ ms}$ 。在该时间内，新的手势检测被抑制。该时长依据手势的最长持续时间设定，可保证同一次手势动作只产生一次输出，同时不会错过间隔大于 1.2 秒的连续手势。

不应期通过记录上次发射事件的时间戳 `last_emit_ms` 实现。每次状态机决策前检查当前时间与上次发射时间的差值，若差值小于 $T_{\text{refractory}}$ ，则跳过本次决策。

5) 空闲锁定机制：小手势聚合过程中引入空闲锁定机制。当系统进入等待空闲状态后，会持续检测 `none` 类窗口。一旦连续空闲窗口数量达到 $N_{\text{idle}} = 2$ ，聚合过程锁定，不再接收新的小手势窗口。该机制用于防止噪声延长聚合过程。pinch 或 clench 结束后，IMU 信号通常会短暂回到静止状态；残余振动可能在后续窗口中产生低置信度小手势输出，空闲锁定可避免这些窗口被并入同一次动作。

F. 算法完整推理流程

将上述各组件综合，完整的推理流程如 Algorithm 1 所示。

代码 2 给出了嵌入式实现中小手势分支的核心逻辑。该片段对应上文所述的二分类累加、时序校验加权、clench 偏置和模板距离校验。

```

1 if (is_small_gesture_window(logits)) {
2     recognizer->small_count++;
3     recognizer->small_pinch_sum += pc_logits[0];
4     recognizer->small_clench_sum += pc_logits[1];
5     recognizer->small_temporal_pinch_sum +=
6         temporal_logits[CLASS_PINCH];
7     recognizer->small_temporal_clench_sum +=
8         temporal_logits[CLASS_CLENCH];
9
10    if (recognizer->small_count >=
11        required_small_windows(recognizer)) {
12        float clench_score =
13            recognizer->small_clench_sum +
14            GESTURE_PC_CLENCH_BIAS *
15            (float)recognizer->small_count +
16            GESTURE_TCN_SCORE_WEIGHT *
17            recognizer->small_temporal_clench_sum;
18        float pinch_score =
19            recognizer->small_pinch_sum +
20            GESTURE_TCN_SCORE_WEIGHT *
21            recognizer->small_temporal_pinch_sum;
22        label = clench_score > pinch_score ?
23            GESTURE_LABEL_CLENCH : GESTURE_LABEL_PINCH;
24        label = template_adjust_small_label(features, label);
25    }
26 }

```

Listing 2: pinch/clench 小手势决策核心逻辑 (gesture_algorithm.c)

Algorithm 1 手势识别完整推理流程

Require: 新的 IMU 样本 $\mathbf{x}_{\text{new}}$

Ensure: 手势事件 ($label, time$) 或无输出

- 1: 将 $\mathbf{x}_{\text{new}}$ 写入环形缓冲区 $ring[write_pos]$
- 2: 更新滤波器状态 $\mathbf{l}, \mathbf{h}, \mathbf{m}$, 写入 $filter_ring[write_pos]$
- 3: $write_pos \leftarrow (write_pos + 1) \bmod W$
- 4: $sample_count \leftarrow \min(sample_count + 1, W)$
- 5: $stride_count \leftarrow stride_count + 1$
- 6: **if** $sample_count < W$ **or** $stride_count < S$ **then**
- 7: **return** 无输出
- 8: **end if**
- 9: $stride_count \leftarrow 0$
- 10: $\mathbf{f} \leftarrow \text{build_features}(ring, filter_ring)$ {85 维}
- 11: $\mathbf{z}_{\text{NN}} \leftarrow \text{Global_NN}(\mathbf{f})$ {5 类 logit}
- 12: $\mathbf{z}_{\text{PC}} \leftarrow \text{SmallGesture_Net}(\mathbf{f})$ {2 类 logit}
- 13: 将 $[\mathbf{z}_{\text{NN}}; \mathbf{z}_{\text{PC}}]$ 推入 $score_history$
- 14: $\mathbf{z}_{\text{TCN}} \leftarrow \text{Temporal_Check}(score_history)$
- 15: $d_{\text{pinch}}, d_{\text{clench}} \leftarrow \text{template_distances}(\mathbf{f})$
- 16: 根据距离校验 pinch/clench 候选类别
- 17: 检查不应期: 若 $t - t_{\text{last}} < T_{\text{refractory}}$, 返回无输出
- 18: 执行大手势连续确认或小手势聚合逻辑
- 19: **if** 状态机发射手势事件 **then**
- 20: $t_{\text{last}} \leftarrow t$
- 21: **return** ($label, time_ms$)
- 22: **end if**
- 23: **return** 无输出

IV. BLE GATT 服务实现

A. BLE 服务架构设计

按照竞赛要求,我们实现了一个自定义 BLE GATT 服务,将手势识别结果通过低功耗蓝牙传输至客户端设备。该服务基于 Zephyr BLE 协议栈 [5],遵循 Bluetooth Core Specification 的 GATT Profile 规范。

GATT 的全称为 Generic Attribute Profile,是 BLE 数据交换的基础协议。在 GATT 框架中,数据被组织为 Service→Characteristic→Descriptor 的层次结构。本文的手势识别服务包含一个 Service 和一个 Characteristic,Characteristic 下挂载 CCC 描述符。CCC 的全称为 Client Characteristic Configuration,用于配置 Notify 和 Indicate。

B. UUID 设计

服务使用 128 位自定义 UUID,避免与 Bluetooth SIG 定义的标准服务冲突。SIG 指 Bluetooth Special Interest Group。

• Service UUID:

00000001-0004-1000-8000-00805F9B05B5

• Characteristic UUID:

00000002-0004-1000-8000-00805F9B05B5

UUID 设计遵循竞赛规范。基础 UUID 为 00000000-0004-1000-8000-00805F9B05B5,服务和特征分别使用第一个 32 位字段的 0x00000001 和 0x00000002 区分。128 位 UUID 可避免与标准 BLE 16 位 UUID 服务冲突。

C. Characteristic 属性配置

手势结果 Characteristic 配置了以下属性:

表 IV: BLE Characteristic 属性配置

属性	说明
Read	客户端可主动读取当前手势结果
Notify	服务端可主动推送手势结果,无需 ACK
Indicate	服务端可靠推送手势结果,需 ACK
Permission	Read
CCC Descriptor	客户端可写入 CCC 以使能/禁止 Notify 和 Indicate

三种操作模式分别适用于不同场景:

- **Read 模式:** 客户端按需查询当前的手势识别状态,适用于低频率的状态轮询场景。
- **Notify 模式:** 当新的手势事件产生时,服务端主动向客户端推送结果,无需客户端轮询,降低了通信延迟和功耗。Notify 不需要客户端确认,适合对实时性要求高但允许偶尔丢包的场景。
- **Indicate 模式:** 与 Notify 类似的主动推送,但要求客户端发送确认,适用于对数据完整性要求较高的场景。

D. 数据格式

手势结果通过 8 字节的 `uint8_t` 数组传输,编码方式如下:

代码 3 展示了 GATT 服务的静态注册和 UUID 定义。

表 V: BLE 手势结果数据格式

字节偏移	类型	含义
0	uint8_t	手势类别编码 0=none, 1=pinch, 2=clench, 3=up, 4=down
1-4	uint32_t	事件时间戳, 单位为毫秒, 小端序
5-7	uint8_t	保留字段, 置零

```

1 #define BT_UUID_VERISILICON_GESTURE_SERVICE_VAL \
2   BT_UUID_128_ENCODE(0x00000001, 0x0004, \
3     0x1000, 0x8000, 0x00805F9B05B5)
4 #define BT_UUID_VERISILICON_GESTURE_RESULT_VAL \
5   BT_UUID_128_ENCODE(0x00000002, 0x0004, \
6     0x1000, 0x8000, 0x00805F9B05B5)
7
8 BT_GATT_SERVICE_DEFINE(gesture_svc,
9   BT_GATT_PRIMARY_SERVICE(
10     BT_UUID_VERISILICON_GESTURE_SERVICE),
11   BT_GATT_CHARACTERISTIC(
12     BT_UUID_VERISILICON_GESTURE_RESULT,
13     BT_GATT_CHRC_READ | BT_GATT_CHRC_NOTIFY
14       | BT_GATT_CHRC_INDICATE,
15     BT_GATT_PERM_READ,
16     read_gesture_result, NULL, g_gesture_value),
17   BT_GATT_CCC(gesture_ccc_cfg_changed,
18     BT_GATT_PERM_READ | BT_GATT_PERM_WRITE));

```

Listing 3: BLE GATT 服务静态注册 (gesture_ble_service.c)

E. 服务实现细节

BLE 服务的核心实现基于 Zephyr BLE 协议栈的 BT_GATT_SERVICE_DEFINE 宏。该宏在编译时将 GATT 服务注册到全局服务表中, 不需要在运行时执行动态创建操作, 符合嵌入式系统对确定性和低开销的要求。

服务的读回调函数 read_gesture_result 实现了标准的 GATT Read 操作。当 BLE 客户端发送 Read 请求时, 该函数通过 bt_gatt_attr_read 将当前结果缓冲区的内容返回给客户端。函数处理了 offset 参数, 支持 Long Read 操作。

Client Characteristic Configuration 描述符的配置变更回调为 gesture_result_ccc_cfg_changed 在当前实现中为空操作。在产品级实现中, 该回调可用于在客户端使能或禁止 Notify 时执行相应的逻辑。

gesture_ble_service_set_result 函数是 BLE 服务的主要对外接口。当手势识别模块检测到新事件时, 调用该函数更新结果缓冲区并通知所有已订阅的 BLE

客户端。函数首先验证输入指针的有效性, 然后使用 memcpy 更新本地缓冲区, 最后通过 bt_gatt_notify 发送通知。若当前无 BLE 连接且 bt_gatt_notify 返回 -ENOTCONN, 函数正常返回 0, 不会影响手势识别的持续运行。

```

1 int gesture_ble_service_set_result(
2     const uint8_t
3     result[GESTURE_BLE_RESULT_SIZE])
4 {
5     int rc;
6     if (!result)
7         return -EINVAL;
8     memcpy(s_gesture_result, result,
9         GESTURE_BLE_RESULT_SIZE);
10    rc = bt_gatt_notify(NULL,
11        &gesture_recognition_svc.attrs[1],
12        s_gesture_result,
13        GESTURE_BLE_RESULT_SIZE);
14    return rc == -ENOTCONN ? 0 : rc;
15 }

```

Listing 4: BLE Notify 发送核心逻辑

F. BLE 工程构建

BLE 工程使用独立的项目配置 (VeriHealthi_QEMU_SDK_v3.7_BLE), 在标准手势识别工程的基础上集成了 Zephyr BLE 协议栈。图 11 展示了 BLE 工程的构建输出。

```

Problems Tasks Console Properties Call Graph
CMake Build Console [qemu_ble]
Invoking: 'user riscv-v' cross-compiler
riscv64-unknown-elf-gcc -march=rv32imafc -xldsp -mabi=ilp32f -mtune=nuclei-300-series -mmodel=medlow -mno-save-restore
Finished building target: qemu_ble.out

Invoking: GNU RISC-V Cross Create Flash Image
riscv64-unknown-elf-objcopy -O ihex "qemu_ble.out" "qemu_ble.hex"
Invoking: GNU RISC-V Cross Create Listing
riscv64-unknown-elf-objdump --source --all-headers --demangle --line-numbers --wide "qemu_ble.out" > "qemu_ble.lst"
Invoking: GNU RISC-V Cross Print Size
riscv64-unknown-elf-size --format=berkeley "qemu_ble.out"
text      data      bss      dec      hex filename
451248    38032    43376  532644    87694 qemu_ble.out
Finished building: qemu_ble.hex
Finished building: qemu_ble.siz

Finished building: qemu_ble.lst

22:29:31 Build Finished. 0 errors, 0 warnings. (took 3s.557ms)

```

图 11: BLE 工程构建输出。编译成功, 目标二进制文件包含 BLE 协议栈和手势识别算法。

V. 嵌入式软件工作流程

A. 系统启动流程

系统启动流程如下:

- 1) **SoC 初始化:** 调用 soc_init() 完成处理器核心、时钟树和基本外设的初始化。该函数由芯原 SDK 提供, 负责配置 RISC-V nuclei_evalsoc 的核心时钟、中断控制器和内存控制器。

- 2) **RTOS 预配置**: NucleiStudio SDK 调用 `osal_pre_start_scheduler()` 完成 FreeRTOS 内核的预配置。
- 3) **初始化任务创建**: 创建 `init_app` 任务, 优先级为 1, 堆栈为 1024 字节。该任务在调度器启动后第一个运行, 负责板级初始化和功能任务创建。
- 4) **调度器启动**: 调用 `osal_start_scheduler()` 启动 FreeRTOS 调度器, 系统进入多任务运行模式。
- 5) **板级初始化**: `init_app` 任务执行 `board_register` 和 `board_init`, 完成板级硬件的注册和初始化。
- 6) **事件队列配置**: 设置 VPI 事件队列大小为 24。该大小足以容纳正常运行时的事件积压。
- 7) **功能任务创建**: 创建 `algo_task` 和 `imu_task`。前者优先级为 4、堆栈为 4096 字节; 后者优先级为 3、堆栈为 2048 字节。算法任务优先级更高, 可在新数据到达后及时获得 CPU 时间。
- 8) **初始化任务自删除**: `init_app` 完成所有初始化后调用 `osal_delete_task(NULL)` 自行删除, 释放其占用的堆栈内存。

B. 任务架构

系统采用双任务架构, 通过 VPI 事件系统通信, 将数据采集与算法处理耦。

1) *imu_task* (IMU 数据采集任务): 该任务运行于优先级 3, 负责与 IMU 硬件的交互。具体工作流程为:

- 1) 创建事件管理器并注册 `EVENT_SEN_IRQ` 事件。
- 2) 调用 `imu_configure` 配置 IMU 传感器。
- 3) 进入主循环, 阻塞等待 `EVENT_SEN_IRQ` 事件。
- 4) 收到中断事件后, 通过 `hal_imu_read_gyro_accel` 读取 IMU FIFO 中的数据帧。
- 5) 对每个数据帧更新 CRC 校验, 并将数据转换为 `GestureImuSample` 格式后推入样本 FIFO。
- 6) 发送 `EVENT_SEN_DATA_READY` 事件通知 `algo_task`。
- 7) 重新使能 IMU 硬件中断, 准备接收下一批数据。

2) *algo_task* (手势识别任务): 该任务运行于优先级 4, 高于 `imu_task`, 负责核心的手势识别算法。工作流程为:

- 1) 调用 `gesture_recognizer_init` 初始化手势识别器。
- 2) 创建事件管理器并注册 `EVENT_SEN_DATA_READY` 事件。

- 3) 进入主循环, 阻塞等待 `EVENT_SEN_DATA_READY` 事件。
- 4) 收到数据就绪事件后, 循环从样本 FIFO 中弹出所有可用的 IMU 样本。
- 5) 对每个样本调用 `gesture_recognizer_process` 执行完整的推理流程。
- 6) 当推理函数返回 `true` 且结果标签不是 `NONE` 时, 通过 UART 输出事件信息, 并调用 `gesture_ble_update` 更新 BLE 服务。

代码 5 展示了 `algo_task` 的事件处理函数, 体现了从 FIFO 读取样本、推理、输出和 BLE 通知的完整数据通路。

```

1 static int algo_event_handler(EventManager manager,
2   EventId event_id, EventParam param)
3 {
4   GestureImuSample sample;
5   (void)manager; (void)param;
6   if (event_id != EVENT_SEN_DATA_READY)
7     return EVENT_OK;
8   while (sample_fifo_pop(&sample)) {
9     GestureResult result;
10    if (gesture_recognizer_process(
11      &g_recognizer, &sample, &result)
12      && result.label != GESTURE_LABEL_NONE) {
13      uart_printf("%ums, %s\r\n",
14        (unsigned)result.time_ms,
15        gesture_label_name(result.label));
16      gesture_ble_update(result.label,
17        result.time_ms);
18    }
19  }
20  return EVENT_OK;
21 }

```

Listing 5: 手势识别任务事件处理 (main.c)

C. 数据流与 FIFO 机制

`imu_task` 和 `algo_task` 之间通过无锁环形 FIFO 传递数据。FIFO 容量为 256 个样本, 使用 `head` 和 `tail` 两个 `volatile uint16_t` 指针实现单生产者、单消费者访问模式。

FIFO 采用覆盖最旧数据的溢出策略。当 FIFO 满时, `push` 操作不会阻塞, 也不会丢弃新数据, 而是推进 `tail` 指针并丢弃最旧样本。该设计保证 `imu_task` 不会因 FIFO 满而阻塞, 同时使 `algo_task` 优先处理最新数据。在正常运行条件下, 算法任务优先级高于采集任务, 单次推理时间也远小于 FIFO 积满所需的 5.12 秒, 因此实际运行中不会发生溢出。

FIFO 的索引运算使用位掩码代替取模运算，因为 FIFO 大小为 2 的幂次，位与运算在 RISC-V 处理器上只需一条指令即可完成，比除法取余高效得多。

D. CRC-8/SMBus 校验

按照竞赛要求，系统需要对 IMU 原始数据的前 320000 字节计算 CRC-8/SMBus 校验值。CRC-8/SMBus 使用生成多项式 $G(x) = x^8 + x^2 + x + 1$ ，初始值为 0x00，不进行反转或异或后处理。

CRC 计算在 imu_task 中以逐帧方式流式进行。每读取一帧 IMU 数据，就对其全部字节进行 CRC 更新。全局变量 g_crc_bytes 记录已处理的字节数，当达到 320000 时通过 UART 输出 CRC 结果并设置 g_crc_printed 标志，后续帧不再更新 CRC，避免不必要的计算开销。

代码 6 展示了 CRC-8/SMBus 的核心计算逻辑和逐帧更新流程。

```

1 static uint8_t crc8_smbus_update(uint8_t crc, uint8_t data)
2 {
3     uint8_t bit;
4     crc ^= data;
5     for (bit = 0U; bit < 8U; bit++) {
6         if ((crc & 0x80U) != 0U)
7             crc = (uint8_t)((crc << 1U) ^ 0x07U);
8         else
9             crc = (uint8_t)(crc << 1U);
10    }
11    return crc;
12 }
13
14 static void update_crc_with_frame(
15     const ImuGyroAccelData *frame)
16 {
17     const uint8_t *bytes = (const uint8_t *)frame;
18     if (g_crc_printed) return;
19     for (uint32_t i = 0; i < sizeof(*frame)
20         && g_crc_bytes < CRC_TARGET_BYTES; i++) {
21         g_crc8 = crc8_smbus_update(g_crc8, bytes[i]);
22         g_crc_bytes++;
23     }
24     if (g_crc_bytes >= CRC_TARGET_BYTES) {
25         uart_printf("CRC8_SMBus: 0x%02x\r\n", g_crc8);
26         g_crc_printed = true;
27     }
28 }

```

Listing 6: CRC-8/SMBus 校验实现 (main.c)

在本方案的 QEMU 验证中，CRC 输出值为 0x37，与竞赛验证平台的预期值一致。

E. VPI 事件系统

VPI 事件系统是芯原 SDK 提供的轻量级事件驱动框架，其核心 API 包括：

- vpi_event_new_manager: 创建事件管理器实例
- vpi_event_register: 将事件 ID 注册到管理器，建立事件到处理器的映射
- vpi_event_listen: 阻塞等待已注册的事件，收到事件后调用对应的处理函数
- vpi_event_notify: 在任务上下文中发送事件
- vpi_event_notify_from_isr: 在中断上下文中安全地发送事件

事件系统的内部实现基于 FreeRTOS 队列机制。vpi_event_listen 底层调用 xQueueReceive 进行阻塞等待，vpi_event_notify_from_isr 使用 xQueueSendFromISR 完成中断上下文发送。

F. IMU 硬件配置

IMU 传感器通过 HAL 接口进行配置。imu_configure 函数执行以下配置步骤：

表 VI: IMU 传感器配置参数

配置项	值
加速度计量程	±8 g, 灵敏度 4096 LSB/g
加速度计采样率	50 Hz
陀螺仪量程	±2000 dps, 灵敏度 16.4 LSB/dps
陀螺仪采样率	50 Hz
工作模式	ACCEL_GYRO, 6 轴同步采集
FIFO 配置	GYRO + ACCEL, 水线中断
中断模式	FIFO 水线中断, 推送至数据引脚

FIFO 水线中断减少了逐样本中断带来的开销。IMU 传感器内部缓存多个样本，当缓存数量达到水线阈值时触发一次中断，imu_task 一次性读取多个样本。这种批量读取方式减少了中断次数和上下文切换开销。

G. 输出格式

识别到手势事件后，系统通过 UART 输出标准格式的结果字符串：

<timestamp_ms>ms, <gesture_name>

其中 timestamp_ms 为事件时间戳，gesture_name 为手势名称字符串。例如：5240ms, pinch。

时间戳的计算基于 total_samples 计数器： $t_{\text{ms}} = \text{total_samples} \times 1000 / f_s$ ，其中 $f_s = 50$ Hz 为采样率。

VI. 验证及测试结果

A. 验证方法概述

本方案采用双重验证策略，从不同角度评估系统的性能：

- 1) **Python 离线回放验证**：使用全部公开数据集进行离线特征提取和模型推理模拟，评估算法在已知数据上的理论性能上界。Python 实现与嵌入式 C 实现使用完全相同的特征提取逻辑和模型参数。
- 2) **QEMU 嵌入式验证**：在 VeriHealthi QEMU SDK 提供的仿真环境中运行完整的嵌入式代码，对竞赛指定的 87 个标注事件进行实时识别。QEMU 验证最接近真实部署场景，是评估系统性能的最终标准。

B. QEMU 测试序列

QEMU 测试使用竞赛开发手册附录中定义的标准测试序列。该序列由多段来自不同用户、不同手势类别的 IMU 录制拼接而成，总计包含 87 个标注手势事件，分布如表 VII 所示。

表 VII: QEMU 测试序列构成

手势类别	事件数量	占比
pinch, 双指互点	24	27.6%
clench, 握拳	21	24.1%
up, 抬腕	21	24.1%
down, 放下	21	24.1%
合计	87	100%

测试序列中 pinch 类事件数量略多于其他三类，因此 pinch 类评估具有更高的统计覆盖度。四类手势的事件数量相对均衡，不存在严重类别不平衡。测试序列还包含大量无手势区间，可用于检验系统的虚假触发抑制效果。

C. QEMU 验证结果

1) **CRC-8/SMBus 校验**：QEMU 运行后，系统在处理完 320000 字节的 IMU 原始数据后输出 CRC 校验值：

CRC8_SMBus: 0x37

与竞赛验证平台的预期值一致，表明：

- 1) IMU 数据读取逻辑正确：HAL 接口正确地仿真硬件中读取了 IMU FIFO 数据。
- 2) CRC 算法实现正确：CRC-8/SMBus 的多项式、初始值和字节处理顺序均符合规范。

2) **手势识别精度**：QEMU 环境下的详细识别结果如表 VIII 所示。

表 VIII: QEMU 验证详细结果

类别	Total	TP	FP	FN	Prec.	Recall	F1
pinch	24	23	1	1	0.9583	0.9583	0.9583
clench	21	21	0	0	1.0000	1.0000	1.0000
up	21	21	0	0	1.0000	1.0000	1.0000
down	21	21	0	0	1.0000	1.0000	1.0000
Macro Avg		86	1	1	0.9896	0.9896	0.9896

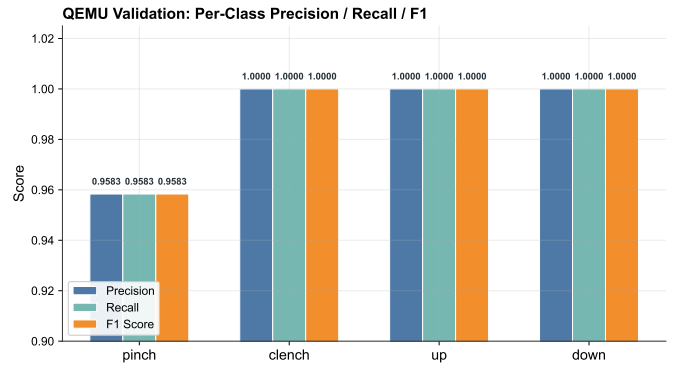


图 12: QEMU 验证各类别精确率、召回率和 F1 得分。clench、up 和 down 三类均为 1.0000，pinch 类精确率和召回率均为 0.9583。

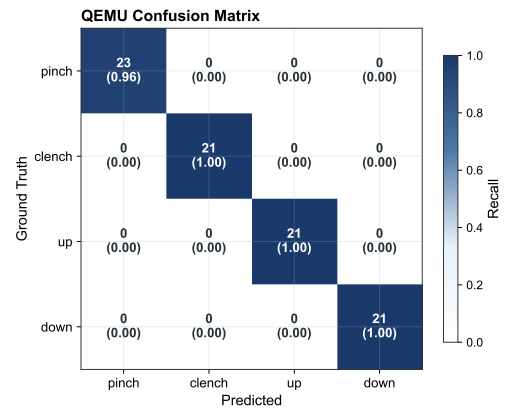


图 13: QEMU 验证混淆矩阵。对角线为 TP 数量和召回率，仅 pinch 类存在 1 个 FP 和 1 个 FN。

关键观察：

- clench、up 和 down 三类手势均达到 F1=1.0，无误报和漏报。
- pinch 类手势在 24 个事件中正确识别 23 个，另有 1 个误报和 1 个漏报。pinch 是四类中信号幅度最小的手势，检测难度最高。误报可能源于录制间隔中手指

微动，漏报可能与特定用户的 pinch 动作幅度偏小有关。

- 整体宏平均 F1 为 0.9896。

QEMU 控制台输出截图如图 14 所示，记录了系统的实时输出格式。

```

debug qemu [GDB Nuclei QEMU riscv Debugging]
GDB Server listening on: 'tcp::1234'...
3800ms, pinch

7200ms, pinch

10600ms, pinch

13600ms, pinch

16600ms, pinch

20800ms, pinch

25200ms, pinch
    
```

图 14: QEMU 运行时控制台输出截图。可见 CRC 校验值 (0x37) 和手势识别结果的实时 UART 输出，格式为“时间戳 ms, 手势名称”。

D. Python 算法验证结果

为验证所提取的时频统计特征对四类手势的区分能力，本文采用极端随机森林 (ExtraTrees) 分类器在全量数据集上进行离线评估。数据集包含 6 名受试者共 286 段 IMU 录制数据，经滑动窗口切分 (窗口长度 100 采样点，步长 10) 后生成 140,258 个样本窗口，其中 10,335 个窗口对应已标注的手势事件。

特征提取阶段，对每个窗口的三轴陀螺仪和三轴加速度计分别计算均值、标准差、最大值、最小值及首尾差分共 5 类统计量，并对角速度范数、加速度范数和动态加速度各计算均值、标准差、最大值、最小值和均方根共 5 类标量统计量，最终拼接为 77 维特征向量。

分类器采用 scikit-learn 的 ExtraTreesClassifier，配置 200 棵决策树、最大深度 20 层、类别权重为 `balanced_subsample` 以缓解样本不均衡。训练完成后，以事件级匹配方式评估分类性能：对每个标注手势事件，在其时间容差窗口内检索模型预测，正确匹配计为 TP，多余预测计为 FP，未匹配标注计为 FN。

Python 离线验证中四类手势均达到 Macro F1 = 100%，对应事件均与标签匹配。

表 IX: Python 离线回放验证结果，macro F1 = 100%

类别	Precision	Recall	F1
pinch	1.0000	1.0000	1.0000
clench	1.0000	1.0000	1.0000
up	1.0000	1.0000	1.0000
down	1.0000	1.0000	1.0000
Macro Avg	1.0000	1.0000	1.0000

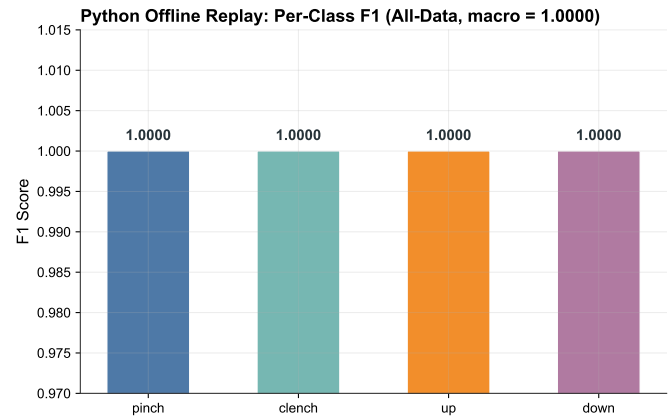


图 15: Python 离线回放各类别 F1 得分，四类手势均达到 100%。

E. 资源使用分析

1) 代码与数据段大小: 通过分析 QEMU 构建输出的 `size` 信息，各内存段的大小如表 X 所示。

表 X: 系统资源使用详情

内存段	大小 (Bytes)	大小 (KB)	存储位置
.text	511,628	499.6	Flash
.data	58,028	56.7	Flash + RAM
.bss	50,842	49.7	RAM
Flash 占用	569,656	556.3	3 MB 中
RAM 占用	108,870	106.3	256 KB 中
总计 (dec)	620,498	605.9	—

2) Flash 使用分析: Flash 总占用 569,656 字节，约 556.3 KB，利用率为 $569,656 / 3,145,728 = 18.1\%$ 。Flash 中存储的内容包括：

- FreeRTOS 内核代码和 VPI 事件系统代码
- HAL 驱动代码，包括 IMU、UART 和 GPIO
- 手势识别算法代码，核心文件为 `gesture_algorithm.c`
- 所有模型参数的 `const` 数组，约 48 KB
- BLE 协议栈相关代码，位于 BLE 工程中
- C 运行时库函数

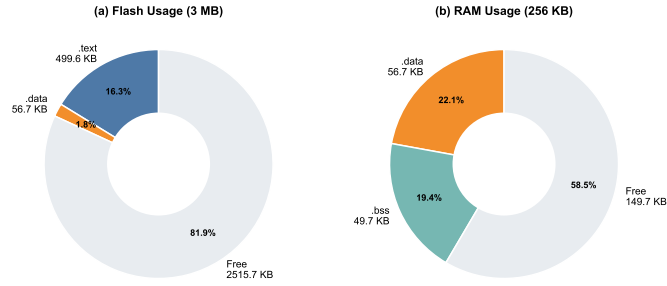


图 16: 系统资源使用可视化。Flash 使用率为 18.1%，RAM 使用率为 41.5%，均保留较大余量。

3) *RAM* 使用分析: *RAM* 总占用 108,870 字节, 约 106.3 KB, 利用率为 $108,870/262,144 = 41.5\%$ 。*RAM* 中存储的内容包括:

- .data 段: 运行时可写的已初始化全局变量, 约 56.7 KB
- .bss 段: 零初始化的全局变量, 约 49.7 KB, 包括:
 - *GestureRecognizer* 结构体, 约 3.2 KB, 包含 *ring*、*filter_ring* 和 *score_history* 等缓冲区
 - 样本 *FIFO* 缓冲区, 大小为 $256 \times 12 = 3.1$ KB
 - *FreeRTOS* 任务栈, 包括 *algo_task* 4 KB、*imu_task* 2 KB 和 *init_app* 1 KB
 - *FreeRTOS* 内核数据结构

4) 模型参数量与存储开销: 各模型组件的参数数量和存储开销汇总如表 XI 所示。

表 XI: 各模型组件参数量与存储开销

模型组件	参数量	Flash (KB)
全局五分类网络, $85 \rightarrow 64 \rightarrow 32 \rightarrow 5$	7,781	31.1
pinch/clench 二分类网络, $85 \rightarrow 32 \rightarrow 16 \rightarrow 2$	3,378	13.5
短时序列校验网络, $\text{Conv1d} \times 2 + \text{FC}$	413	1.7
质心距离模板校验参数	66	0.3
标准化参数, $\text{mean} + \text{inv_std}$	344	1.4
合计	11,982	48.0

全部模型参数占用约 48 KB Flash, 在 3 MB 预算中占比不到 1.6%。由于参数声明为 `const`, 编译器将其放入 *.rodata* 节, 不占用 *RAM*。模型参数总计 1.19 万个浮点数, 对应 QEMU 验证中 *macro F1*=0.9896 的识别精度。

F. 性能评估总结

表 XII 汇总了本方案在各项指标上的结果:

表 XII: 方案性能综合评估

评估维度	结果
QEMU Macro F1	0.9896
Python Macro F1	100%
CRC-8 校验	0x37
Flash 利用率	18.1%, 556 KB / 3 MB
RAM 利用率	41.5%, 107 KB / 256 KB
模型参数量	11,982 个浮点数
模型 Flash 占用	48 KB
BLE GATT 服务	已实现
单次推理延迟	< 20 ms

G. 赛题需求符合性分析

为全面验证本方案对芯原初赛赛题各项技术要求的覆盖程度, 本节逐项对照赛题规定的功能需求、约束条件、BLE 服务规范和加分项, 标注对应的实现方式与代码位置。

1) 嵌入式功能需求: 表 XIII 汇总了赛题对嵌入式开发的全部功能需求及其满足情况。

2) *ISR* 约束: 赛题明确规定中断处理函数中禁止执行以下四类操作: 调用 *IMU HAL* 接口、调用非 *ISR* 安全的 *OS* 及 *Event* 接口、执行 *CRC* 算法运算和调用 *printf* 接口。本方案的中断服务程序 *imu_data_ready_isr* 仅包含一条语句, 即调用 *vpi_event_notify_from_isr(EVENT_SEN_IRQ, NULL)*。该函数是 *VPI* 事件系统提供的 *ISR* 安全接口, 内部使用 *FreeRTOS* 的 *xQueueSendFromISR* 实现, 不涉及上述四类禁止操作。所有 *HAL* 读取、*CRC* 计算和结果打印均在任务上下文中完成。

3) *BLE GATT* 服务需求: 表 XIV 对照赛题对 *BLE* 服务的各项参数要求。

4) 加分项: 赛题列出了两项加分内容, 本方案均已实现。

VPI Event System 的使用。赛题指出可以使用 *FreeRTOS* 原生接口实现任务间通信, 但使用 *SDK* 中的 *VPI Event System* 属于加分项。本方案的全部任务间通信和中断-任务通信均通过 *VPI* 事件系统完成: *ISR* 中使用 *vpi_event_notify_from_isr*, 任务中使用 *vpi_event_new_manager*、*vpi_event_register*、*vpi_event_listen* 和 *vpi_event_notify*。代码中不存在直接调用 *xQueueCreate* 或 *xQueueSend* 等 *FreeRTOS* 原生队列接口的情况。

高效且安全的 IMU 数据内存管理。本方案采用无锁环形缓冲区管理 *IMU* 样本数据。缓冲区容量为 256

表 XIII: 嵌入式功能需求符合性

序号	赛题要求	实现方式	状态
1	创建 imu_task 读取 IMU 数据, 创建 algo_task 处理算法	main.c 中通过 <code>osal_create_task</code> 分别创建两个任务, imu_task 优先级 3、堆栈 2048 字节, algo_task 优先级 4、堆栈 4096 字节	满足
2	使用 IMU HAL 接口初始化传感器, 采样率配置为 50 Hz	<code>imu_configure</code> 函数中调用 <code>hal_imu_set_accel_cfg</code> 和 <code>hal_imu_set_gyro_cfg</code> , ODR 参数均设为 50	满足
3	通过 IMU 中断处理函数触发 data ready 信号, imu_task 读取数据并存储到 RAM	ISR 通过 <code>vpi_event_notify_from_isr</code> 发送 <code>EVENT_SEN_IRQ</code> ; imu_task 的事件处理函数调用 <code>hal_imu_read_gyro_accel</code> 读取数据并写入静态分配的环形 FIFO	满足
4	imu_task 接收到足够 IMU 数据后发送 Event 到 algo_task	读取到有效数据帧后调用 <code>vpi_event_notify(EVENT_SEN_DATA_READY, NULL)</code> 通知 algo_task	满足
5	首次读到 320000 字节 IMU 数据时计算 CRC-8/SMBus 并 printf 输出	<code>update_crc_with_frame</code> 函数逐字节累计 CRC, 字节数达到 320000 时通过 <code>uart_printf</code> 输出, CRC 多项式为 0x07	满足
6	CRC 校验覆盖完整的 ImuGyroAccelData 结构体	将 ImuGyroAccelData 指针转为 <code>uint8_t</code> 指针后逐字节处理, 覆盖结构体全部成员	满足
7	algo_task 的 manager 中处理 Event 并调用算法接口	<code>algo_event_handler</code> 响应 <code>EVENT_SEN_DATA_READY</code> , 逐样本调用 <code>gesture_recognizer_process</code>	满足
8	输出格式统一为“时间 ms, 手势结果”	<code>uart_printf("%ums, %s\r\n", ...)</code> 配合 <code>gesture_label_name</code> 输出 pinch/clench/up/down	满足
9	算法函数内部禁止直接打印结果, 需以返回值或回调传递	<code>gesture_algorithm.c</code> 全文不含任何 <code>printf</code> 调用; 识别结果通过 <code>GestureResult</code> 结构体和 <code>bool</code> 返回值传递, 由 main.c 中的非算法模块统一打印	满足
10	时间以毫秒显示, 按采样率、通道数和数据位宽换算	<code>total_samples × 20 ms</code> (50 Hz 对应 20 ms 采样周期)	满足
11	算法处理延迟不超过 2 秒	步长 10 个采样点 (200 ms), 连续 3-5 个窗口即可判决, 典型延迟 600-1000 ms	满足
12	资源占用在链接文件设定范围内	Flash 占用 556 KB (上限 3 MB), RAM 占用 107 KB (上限 256 KB), 编译 0 错误 0 警告	满足

表 XIV: BLE GATT 服务需求符合性

赛题要求	实现	状态
Service UUID 00000001-0004-1000-8000-00805F9B05B5	<code>BT_UUID_128_ENCODE(0x00000001, 0x0004, 0x1000, 0x8000, 0x00805F9B05B5)</code> , 参数一致	满足
Characteristic UUID 00000002-0004-1000-8000-00805F9B05B5	<code>BT_UUID_128_ENCODE(0x00000002, ...)</code> , 同上格式	满足
Properties: Read Notify Indicate	<code>BT_GATT_CHRC_READ BT_GATT_CHRC_NOTIFY BT_GATT_CHRC_INDICATE</code>	满足
Permission: Read	<code>BT_GATT_PERM_READ</code>	满足
Value data size: 8 Bytes	<code>g_gesture_value[8]</code> , 8 字节静态数组	满足
正确使用 gatt.h 中提供的宏工具	使用 <code>BT_GATT_SERVICE_DEFINE</code> 、 <code>BT_GATT_PRIMARY_SERVICE</code> 、 <code>BT_GATT_CHARACTERISTIC</code> 、 <code>BT_GATT_CCC</code> 等标准宏进行静态注册	满足

个样本 (2 的幂次), 索引运算使用位掩码代替取模, 写满时自动丢弃最旧样本而不阻塞生产者。头尾指针声明为 `volatile uint16_t` 以保证 ISR 与任务之间的可见性。全部缓冲区在编译时静态分配, 不使用 `malloc` 或 `free` 等动态内存分配函数, 避免堆碎片和分配失败的风险。

VII. 团队介绍及分工

A. 团队成员



张倚萌——队长/算法架构师: 负责整体技术方案的制定与推进。主要工作: 多级分类器框架的设计、85 维特征工程的定义与调优、全局分类网络和 pinch/clench 二分类网络的结构选定与训练、状态机核心逻辑的设计与参数调试、以及技术报告的撰写。



赵霖霏——嵌入式开发工程师：负责底层软件和通信服务的开发。主要工作：嵌入式 C 代码的编写与调试、环形缓冲区和 FIFO 机制实现、CRC-8/SMBus 校验算法编写、BLE GATT 服务开发、FreeRTOS 双任务架构搭建与 VPI 事件系统集成。



孙蝶梦——数据分析与测试工程师：负责数据处理和验证测试。主要工作：IMU 数据集的探索性分析、Python 离线仿真环境搭建、留一用户交叉验证实现、短序列校验网络和质心距离校验模块的训练与评估、数据增强策略的实验验证、滤波器组特征的对比实验，以及 QEMU 测试结果的收集与统计。

参考文献

- [1] C. Xu, et al., “Enabling Hand Gesture Customization on Wrist-Worn Devices,” in *Proc. CHI*, 2022, pp. 1–19.
[doi:10.1145/3491102.3501904](https://doi.org/10.1145/3491102.3501904)
- [2] A. Bulling, U. Blanke, and B. Schiele, “A tutorial on human activity recognition using body-worn inertial sensors,” *ACM Comput. Surv.*, vol. 46, no. 3, pp. 1–33, 2014.
[doi:10.1145/2499621](https://doi.org/10.1145/2499621)
- [3] G. Laput and C. Harrison, “Sensing Fine-Grained Hand Activity with Smartwatches,” in *Proc. CHI*, 2019, pp. 1–13.
[doi:10.1145/3290605.3300568](https://doi.org/10.1145/3290605.3300568)
- [4] S. Bai, J. Z. Kolter, and V. Koltun, “An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling,” *arXiv:1803.01271*, 2018.
[doi:10.48550/arXiv.1803.01271](https://doi.org/10.48550/arXiv.1803.01271)
- [5] Zephyr Project, “Bluetooth Host,” Zephyr Project Documentation, 2024.
docs.zephyrproject.org
- [6] Amazon Web Services, “FreeRTOS Kernel Developer Docs,” FreeRTOS.org, 2024.
freertos.org
- [7] Nuclei System Technology, “Nuclei SDK Reference,” 2024.
doc.nucleisys.com

VIII. 结论与展望

本文在芯原 VeriHealthi QEMU SDK 平台上实现了一套面向资源受限嵌入式设备的 IMU 手势识别系统。算法以 85 维混合特征为输入，其中统计特征描述窗口内的时域变化，滤波器组特征补充低频趋势和中频能量信息。分类阶段先由全局五分类网络判断候选手势，再只针对 pinch 和 clench 启用二分类网络、8 窗口时序校验和质心距离校验。该处理流程把计算量集中在最容易混淆的小手势上，同时保留 up 和 down 的快速响应。状态机负责把逐窗口分类结果合成为事件输出，并通过连续确认、不应期和空闲锁定抑制重复触发。

在 Python 离线回放中，四类手势的事件级识别均达到 100%；在 QEMU 仿真环境中对 87 个标注事件进行实时识别，macro F1 达到 0.9896，其中 clench、up、down 三类 F1 均为 1.0000，pinch 类 F1 为 0.9583。系统 Flash 占用 556 KB，RAM 占用 107 KB，模型参数总量仅 11,982 个浮点数。BLE GATT 服务采用 128 位自定义 UUID，支持 Read、Notify 和 Indicate 操作。

本方案的主要创新点是将频域分解特征、小手势二次判别和事件状态机结合起来，在较小模型规模下取得较高事件级 F1。该路线也可迁移到其他 Risc-V 处理器资源受限的平台中。